Lctr 5: Pointers and Unit Test

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Pointers of C.
  - Definition of pointers.
  - Using array name as pointers.
  - Typecasting pointers or dereferenced values.
  - Offset of pointers.
  - Prefix and postfix increment and decrement of variables and pointers.
- Unit test with Unity.

Demo project:

- Project will have two builds: App and Unit test.
- Unit test will test various pointer operations. It also tests a function for string copy implemented by using pointers and `do-while` loop.
- App will print out various pointer operations as well.

Additional reading:

- Stack and heap.

5.1 Introduction

5.2 Demo project

We will build two executables for each supported MPB in the demo project. **One is for unit test**, and **the other is for a normal application**, the App.

For the unit test, we do the following:

- Test the change of the pointers after simple operations. This is mainly for the understanding of the pointers.
- Test two functions implemented by using pointers and the `do-while` loop.

For the App, we do the following:

- Use the tested functions for string manipulation and other work.

5.3 Memory view, pointers, and array

A **pointer** is a variable holding the address of another variable.

5.3.1 Pointer-related operators and their precedence

Precedence	Operator	Description	Associativity
1	`[ ]`	Array subscripting	Left to right
	`++ --`	Postfix increment and decrement `a++`	
2	`++ --`	Prefix increment and decrement `++a`	Right to left
	`(`*data_type*`)`	Typecasting to *data_type*	
	`*`	Dereferencing a pointer	
	`&`	Taking address of a variable	

`ch = *src++` is the same as `ch = *(src++)`

5.3.2 Variables and their memory view

Here we consider several variables that will be used to experiment with pointers.

Data type	Name	Address	Value
`int16_t`	`v16a`	`0x2000_0000`	`0xBEEF`
`int16_t`	`v16b`	`0x2000_0002`	`0xDEAD`
`int8_t`	`v8a`	`0x2000_0004`	`0x78`
`int8_t`	`v8b`	`0x2000_0005`	`0x56`
`int8_t`	`v8c`	`0x2000_0006`	`0x34`
`int8_t`	`v8d`	`0x2000_0007`	`0x12`

Normally, we do not know where these variables are allocated without modifying the linker file. Here, we just assume we have these variables specifically allocated using a specific linker file. This way, we can link to the memory map, as shown below.

To illustrate the ideas, we use the memory map in Figure 1 to associate with the above variables.

32-bit address	8-bit content
0x2000_0000	0b1110_1111=0xEF
0x2000_0001	0b1011_1110=0xBE
0x2000_0002	0b1010_1101=0xAD
0x2000_0003	0b1101_1110=0xDE
0x2000_0004	0b0111_1000=0x78
0x2000_0005	0b0101_0110=0x56
0x2000_0006	0b0011_0100=0x34
0x2000_0007	0b0001_0010=0x12

Figure 1: Memory view of addresses and contents of a few variables.

5.3.3 Definition and initialization of pointers

A pointer is a special type of variable. We defined a pointer by using the following syntax: `data_type * ptr_name [= address];`.

Here:

- `data_type` is the data type of the accessed.
- `*` is the modifier, signaling the following one is a pointer.
- `ptr_name` is the name of the pointer variable.
- We can *optionally* initialize the pointer by providing a specific address.

All pointers should be initialized before being used. Below is the definition and initialization of a pointer to an `int16_t` variable, where the `&` operator is used to take the address of `v16a`. So, pointer `p_16_var1` holds the address of `v16a`, which is `0x2000_0000`.

```
int16_t *p_16_var1 = &v16a;
```

The definition and initialization of a pointer can be done in two lines of code as well, such as the following:

```
int16_t *p_16_var2;
p_16_var2 = &v16b;
```

Here, pointer `p_16_var2` holds the address of `v16b`, which is `0x2000_0002`.

Data type	Name	Address	Value
int16_t	v16a	0x2000_0000	0xBEEF
int16_t	v16b	0x2000_0002	0xDEAD
int8_t	v8a	0x2000_0004	0x78
int8_t	v8b	0x2000_0005	0x56
int8_t	v8c	0x2000_0006	0x34
int8_t	v8d	0x2000_0007	0x12

temp

5.3.4 Accessing data via pointers

To swap two values using variables:

```
int16_t temp = v16a;
v16a = v16b;
v16b = temp;
```

The above can also be performed using pointers:

```
int16_t temp = *p_16_var1;
*p_16_var1 = *p_16_var2;
*p_16_var2 = temp;
```

Example 1: Consider the memory view of Figure 1. If the 4 bytes saved from `0x2000_2004` to `0x2000_2007` is a (32-bit) word in big-endian, coming from an external device. Write a block of code using the pointers to `int8_t` variables to change it to little-endian base on the following definitions:

```
int8_t *p_8_var1 = 0x2000_2004;
int8_t *p_8_var2 = 0x2000_2005;
int8_t *p_8_var3 = 0x2000_2006;
int8_t *p_8_var4 = 0x2000_2007;
int8_t temp;
```

0x2000_0004	0b0111_1000=0x78
0x2000_0005	0b0101_0110=0x56
0x2000_0006	0b0011_0100=0x34
0x2000_0007	0b0001_0010=0x12

Solution:

```
temp = *p_8_var1;
*p_8_var1 = *p_8_var4;
*p_8_var4 = temp;
temp = *p_8_var2;
*p_8_var2 = *p_8_var3;
*p_8_var3 = temp;
```

:-:

5.3.5 Using array name as a pointer

```
char str1[] = "Hello world.";
char str2[] = "Hi, there.";
```

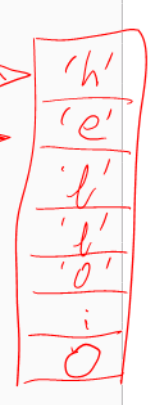
Now, let's take a look at the following operations:

```
str1[0] = 'h';
*str2 = 'h';
```

They both change `'H'` into `'h'`.

Here, we can write the following function to convert each alphabet in a string to upper case while keeping others unchanged.

```
void mp_str2upper(char *src) {
    char ch;
    do {
        ch = *src;
        if (ch >= 'a' && ch <= 'z') {
            *src -= 32; // Convert to upper case
        }
        src = src + 1;
    } while (ch);
}
```



The above code can be changed to using the array index approach as shown below:

```
void mp_str2upper(char *src) {
    int i = 0;
    char ch;
    do {
        ch = src[i];
        if (ch >= 'a' && ch <= 'z') {
            src[i] -= 32; // Convert to upper case
        }
        i++;
    } while (ch);
}
```

5.4 Operations on pointers or dereferenced variables

5.4.1 Specifying typecasting to the pointer or the dereferenced value

The pointer has two aspects, as illustrated by `p_16_var1` we used above:

- The address of the variable it points to, which is `0x2000_0000`.
- The dereferenced value of the pointer, the value of the `int16_t` variable the pointer points to, which is `0xBEEF`.

5.4.1.1 Typecasting one pointer to a pointer of another data type

We can typecast pointer `p_16_var1`, which has a value of `0x2000_0000`, to pointers of two other data types, as shown below:

```
int32_t *p_32_var = (int32_t *)p_16_var1;
int8_t *p_8_var = (int8_t *)p_16_var1;
```

5.4.1.2 Typecasting the dereferenced value to another data type

We can typecast the dereferenced value from `p_16_var1` to values of other data types, as shown below:

```
int32_t v_32 = (int32_t)*p_16_var1;
int8_t v_8 = (int8_t)*p_16_var1;
```

Data type	Name	Address	Value
int16_t	v16a	0x2000_0000	0xBEEF
int16_t	v16b	0x2000_0002	0xDEAD
int8_t	v8a	0x2000_0004	0x78
int8_t	v8b	0x2000_0005	0x56
int8_t	v8c	0x2000_0006	0x34
int8_t	v8d	0x2000_0007	0x12

- `v_32` will have a value of `0xFFFF_BEEF`.
- `v_8` will have a value of `0xEF`.

5.4.1.3 Contrast to two different type-casting operations

Now, we put the two different types of type-casting operations together to better understand them:

```
int32_t *p_32_var1 = (int32_t *)p_16_var1;
int32_t p_32_var2 = (int32_t)*p_16_var1;
```

5.4.2 Offsetting pointers and automatic address change

Assume:

```
int8_t  arr08[] = {1, 2, 3, 4};
int8_t  *pvar08 = arr08;
int32_t arr32[] = {11, 12, 13, 14};
int32_t *pvar32 = arr32;
```

5.4.2.1 Offsetting pointers

```
printf("Content value of arr08[0]: %d\n", *pvar08);
printf("Content value of arr08[2]: %d\n", *(pvar08+2));
printf("Content value of arr32[0]: %ld\n", *pvar32);
printf("Content value of arr32[2]: %ld\n", *(pvar32+2));
}
```

```
Content value of arr08[0]: 1
Content value of arr08[2]: 3
Content value of arr32[0]: 11
Content value of arr32[2]: 13
```

5.4.2.2 Adjusting the memory address automatically according to data type

```
printf("Address of arr08[0]: %p\n", (void *)pvar08);
printf("Address of arr08[2]: %p\n", (void *)(pvar08+2));
printf("Address of arr32[0]: %p\n", (void *)pvar32);
printf("Address of arr32[2]: %p\n", (void *)(pvar32+2));
```

```
Address of arr08[0]: 0x2003ff74
Address of arr08[2]: 0x2003ff76
Address of arr32[0]: 0x2003ff64
Address of arr32[2]: 0x2003ff6c
```

A couple of notes:

- `void *` is the universal pointer type. For memory, it points to bytes.
- `arr08` is an array of 1-byte numbers. An offset of 2 on the pointer corresponds to `0x2003ff76` - `0x2003ff74` = 2 in byte address.
- `arr32` is an array of 4-byte numbers. An offset of 2 on the pointer corresponds to `0x2003ff6c` - `0x2003ff64` = 8 in byte address.

5.4.3 Increment/decrement operations on variables and pointers

See page 12 for an In-class Activity problem for 1/21/26.

5.4.3.1 Prefix/postfix increment/decrement operations on variables

```
void increments_starting_at_zero(int *arr) {
    int i = 0;
    arr[0] = ++i;    // arr[0] = 1 and i = 1 after operation
    arr[1] = i;      // arr[1] = 1 and i = 1 after operation
    arr[2] = i++;    // arr[2] = 1 and i = 2 after operation
    arr[3] = i;      // arr[3] = 2 and i = 2 after operation
}
```

5.4.3.2 Prefix/postfix increment/decrement operations on pointers

```
void mp_strcpy(char *dst, char *src) {
    char ch;
    do {
        ch = *src++;
        *dst++ = ch;
    } while (ch);
}
```

`ch = *src++` is the same as `ch = *(src++)`, which can be considered as

```
ch = *src;
src += 1;
```

5.4.3.3 Increment/decrement operations on dereferenced values of pointers

We have discussed that `ch = *(src++)` is the same as `ch = *src++`, which is the same as

```
ch = *src;
src += 1;
```

used to **dereference the pointer and then increment the pointer**.

Now, what about `ch = (*src)++`?

```
ch = *src;
*src = ch + 1;
```

`Done on 1/21/26`

5.5 Unit test with Unity

We illustrate the concepts of **unit test** using the code snippets defined in `test_ci1s_ptr_n_fns.c`.

5.5.1 Importance of using unit test

5.5.2 Test functions and assertions

To test a specific FUT, we need to write a test function to include all the intended **test cases** for testing it. Each of these test cases is tested with an **assertion** of a specific type. Below are examples of assertions we use often.

5.5.2.1 Assertions for validity tests

```
TEST_ASSERT_TRUE(condition)
```

This assertion evaluates the **code** in `condition`. This assertion fails if the `condition` evaluates to false. For most cases, `condition` is a function we test. The opposite assert is given as

```
TEST_ASSERT_FALSE(condition)
```

Here is an example of a FUT and a test function including a *True* assertion:

```
bool a_no_greater_than_b(int a, int b) {
    return a <= b;
}

void test_a_no_greater_than_b(void) {
    int a = 1, b = 2;
    TEST_ASSERT_TRUE(a_no_greater_than_b(a, b));
}
```

Note that it is a convention that **the name of the test function should convey the information of the FUT**.

5.5.2.2 Assertions for integers

Many functions return a value of the signed integer type. In this case, we can use one of the following assertions.

```
TEST_ASSERT_EQUAL_INT(expected, actual)
TEST_ASSERT_EQUAL_INT8(expected, actual)
TEST_ASSERT_EQUAL_INT16(expected, actual)
TEST_ASSERT_EQUAL_INT32(expected, actual)
TEST_ASSERT_EQUAL_INT64(expected, actual)
```

```
uint32_t double_by_left_shift(uint32_t x) {
    return x << 1;
}

void test_double_by_left_shift(void) {
    uint32_t x = 0x01020304;
    uint32_t exp = 0x02040608;
    uint32_t act = double_by_left_shift(x);
    TEST_ASSERT_EQUAL_UINT32(exp, act);
}
```

We can easily verify that the above test should pass.

In class activity 1/21/26

```
void increments_starting_at_zero(int *arr) {
    int i = 0;
    arr[0] = ++i;    // arr[0] = 1 and i = 1 after operation
    arr[1] = i;      // arr[1] = 1 and i = 1 after operation
    arr[2] = i++;    // arr[2] = 1 and i = 2 after operation
    arr[3] = i;      // arr[3] = 2 and i = 2 after operation
}
```

10 i ++i

--i

5.5.2.3 Assertions for integer arrays

We just need to append `_ARRAY` at the end of any of the integer test assertions discussed above, as shown below for the `int32_t` type:

```
TEST_ASSERT_EQUAL_INT32_ARRAY(expected, actual, elements)
```

Of course, we need to indicate the number of elements in each array we want to test, which is `elements` here.

For example, if we want to test the operations in the `increments_starting_at_zero` function in the Prefix/postfix increment/decrement operations on variables section above, we can use the following test function with the assertion for `uint32_t` array.

```
void test_increments_starting_at_zero(void) {
    int arr_exp[] = {1, 1, 1, 2};   // expected array
    int arr_act[4];                 // actual array
    increments_starting_at_zero(arr_act);
    TEST_ASSERT_EQUAL_INT32_ARRAY(arr_exp, arr_act, 4);
}
```

5.5.2.4 Assertions for other types

There are assertions for other data types or formats. See https://github.com/ThrowTheSwitch/Unity for details.

5.5.3 Test function runner

The above test functions should be called by the test function runners in the *main* test function using the following syntax:

```c
void mp_unity(void) {
    UNITY_BEGIN();

    RUN_TEST(test_a_no_greater_than_b);
    RUN_TEST(test_double_by_left_shift);
    RUN_TEST(test_increments_starting_at_zero);

    // other tests

    UNITY_END();
}
```

Here, `UNITY_BEGIN()` and `UNITY_END()` are used to set up and tear down the tests, and `RUN_TEST` is the test function runner, a macro defined to run each test. Each test function should be run by a test runner.

Here is the running result of this program:

```
/test_ci1s_ptr_n_fns.c:70:test_a_no_greater_than_b:PASS
/test_ci1s_ptr_n_fns.c:71:test_double_by_left_shift:PASS
/test_ci1s_ptr_n_fns.c:72:test_increments_starting_at_zero:PASS
/test_ci1s_ptr_n_fns.c:75:test_mp_str2upper_dowhile:PASS
/test_ci1s_ptr_n_fns.c:76:test_mp_strcpy:PASS
-----------------------
5 Tests 0 Failures 0 Ignored
OK
```

If there are failed tests, the result will clearly indicate the line numbers where the failures happened, which is very useful for us to find the issues.

5.6 Additional reading

This section is mainly about memory. Here the memory only refers to RAM. We will discuss peripheral registers later using pointers for structs later.

5.6.1 Accessibility of variables

- Variables defined outside all functions are called global variables.
- We can use `extern` to make a global variable truly *global*. When we have a global variable defined in a source, we can use this `extern` keyword to declare this variable in other C source files so that we use the same global in different source files.
- If a global is modified by using `static`, it will be a private variable for the file that defines it.
- A local variable of a function can also be modified as `static`. This way, its value will persist between function calls.

5.6.2 Volatility of variables

A *private or global* variable can be modified with the `volatile` keyword as the **volatile** type,

5.6.3 Memory blocks

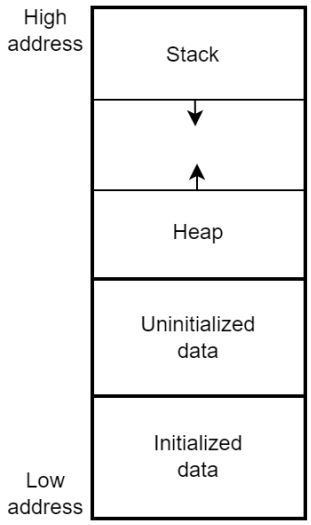


Figure 2: Three memory blocks in RAM: data segment, heap, and stack.